Microservices Patterns

Breaking a complex monolith system into a set of collaborating microservices pays rich dividends in terms of auto-scaling, time-to-market and adopting modern technologies, etc, to name a few. So far, we have seen how a monolith like UMS (user management system) can be decomposed and deployed in a polyglot environment. In this twelfth and last part of the series, we take a look at the patterns that are central to the success of microservices architecture. We conclude with the final design of the UMS.

Microservices are relatively small in size and easy in complexity, in comparison with monolith systems. It doesn't mean that chopping down a complex system into smaller microservices will automatically solve the problem. There are two aspects to microservices. Decomposition is invariably one of them. The domain-driven design lays fundamental principles that govern the decomposition. The success of microservices architecture also depends on the second aspect, i.e., deployment. It deals with how these microservices are collaborated, exposed, monitored and managed.

There are quite a few patterns in the area of microservices that are popular. Tools like Dockers for containerisation and Kubernetes for orchestration implement some of these patterns. For instance, we have seen how easy it is to deploy containerised services like *AddService*, *FindService* and *SearchService* without worrying about draining the resources. We have also seen how Kubernetes is helpful in creating and managing the replicas of these services to scale. Let us have a look at a few more microservices patterns.

Data management patterns

In the case of UMS, we have used only one database for all the microservices. This is referred to as shared database pattern, and is not necessarily always the case. Individual microservices may maintain the data in separate databases. For instance, in an e-commerce environment, *OrderService* may store the orders in one database whereas *InventoryService* may store the product inventory in another database. This approach is called database per service pattern. This doesn't mean that *OrderService* uses MySQL whereas *InventoryService* uses MongoDB. The separation can also be in terms of dedicated schema, dedicated tables or collections, etc. The advantage of *database per service* is obvious. Each service can choose the best data management suitable to its needs. A change in the data model of one service does not impact the other services.

Like in every other sphere, nothing comes for free. *Database per service* poses an issue at the time of processing the data that spans two services. For instance, booking an order involves *OrderService* and something like